

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

4

Capítulo

Gestión de Soluciones - IV

Julio Alberto López Gómez

En este capítulo se profundizará en las **bases de datos orientadas a grafos** como solución a problemas de *Big Data*. En la actualidad, cualquier problema puede ser modelizado por medio de grafos. Esta estructura matemática **versátil y escalable** proporciona un mayor rendimiento a los sistemas que utilizan este tipo de bases de datos. Para ilustrar todos los conceptos de este capítulo, se utilizará **Neo4j** como plataforma de bases de datos orientada a grafos.

4.1. Bases de datos orientadas a grafos

Una base de datos orientada a grafos es un tipo de base de datos **NoSQL** como lo son también, por ejemplo, las bases de datos documentales. Un sistema de gestión de bases de datos orientadas a grafos es aquel **define métodos y operaciones CRUD sobre un modelo de datos representado mediante un grafo**. Por norma general, este tipo de bases de datos se implementan para su uso junto con sistemas OLTP, por lo que están diseñadas en términos de relaciones de integridad y optimizadas para ofrecer un rendimiento operacional.

Aunque existen diferentes tipos de modelos de datos basados en grafos, cada sistema de base de datos elegirá uno de ellos para representar el contenido de la base de datos. Cualquiera de estos modelos está inspirado en **la estructura matemática de un grafo para modelizar un conjunto de datos**. Matemáticamente, un grafo es un conjunto de **nodos o vértices**, que representan entidades de un dominio, y un conjunto de **aristas o enlaces**, que representan las relaciones o interacciones que se producen entre las entidades. La figura 4.1 muestra un ejemplo de grafo que representa las relaciones de seguimiento entre distintos canales en YouTube.

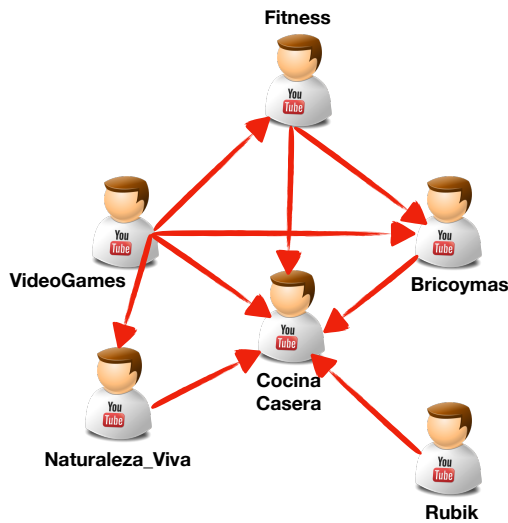


Figura 4.1: Ejemplo de grafo: relaciones de seguimiento entre canales YouTube

Uno de los modelos de datos basado en grafos más popular es **el modelo de grafo de propiedades etiquetadas**. Un grafo representado mediante este modelo, debe cumplir las siguientes características principales: **1)** debe contener un conjunto de nodos y aristas o relaciones entre dichos nodos, **2)** los nodos contienen propiedades, definidas a través de pares “clave-valor”, **3)** los nodos pueden estar etiquetados con una o más etiquetas, **4)** las relaciones entre los nodos están nombradas y son dirigidas, teniendo siempre un nodo de inicio y otro nodo de fin, **5)** las relaciones del grafo también pueden contener propiedades. Este modelo, a pesar de su simplicidad, es sencillo de entender y permite modelar cualquier problema y/o conjunto de datos.

4.1.1. Características principales

El aumento del uso de los sistemas de bases de datos orientados a grafos y el auge que están alcanzando en los últimos años se debe, principalmente, a una serie de características que confieren a estos sistemas una serie de ventajas importantes sobre los sistemas de bases de datos tradicionales y otros sistemas NoSQL.

- **Almacenamiento:** Las bases de datos orientadas a grafo pueden ser de almacenamiento nativo, en cuyo caso los datos se almacenan internamente con estructura de grafo. No obstante, existen bases de datos de grafos que mapean los datos para adaptarlos a una estructura relacional.

- **Procesamiento:** El almacenamiento nativo permite obtener una estructura de datos que no requiere índices de adyacencia para referenciar los nodos del grafo, mejorando el rendimiento en la ejecución de consultas.
- **Rendimiento:** La ejecución de consultas sobre datos estructurados en forma de grafo proporciona escalabilidad al sistema de bases de datos, ya que aunque el número de datos aumente, las consultas se ejecutan sobre la porción del grafo de interés.
- **Flexibilidad:** Los modelos de datos basados en grafos permiten rápidamente añadir nuevos nodos, enlaces y subgrafos a un modelo existente con facilidad y sin pérdida de funcionalidad ni rendimiento.
- **Rapidez:** La naturaleza libre de esquema de las bases de datos orientadas a grafos, junto con las APIs desarrolladas para su manipulación y los lenguajes de consulta, hacen que estos sistemas sean ágiles y rápidos, integrándolos con las metodologías de desarrollo de software iterativas e incrementales.

4.1.2. Áreas de Aplicación

Las características de las bases de datos orientadas a grafos desarrolladas en el apartado anterior confieren a esta tecnología un sinfín de **aplicaciones reales** en las que su uso mejora significativamente a las tecnologías tradicionales. A continuación, se muestran algunos ejemplos de aplicación donde, actualmente, este tipo de bases de datos es el más ampliamente utilizado.

- **Redes sociales:** El análisis de redes sociales es, a día de hoy, una de las principales fuentes de información de cualquier dominio. El análisis de redes sociales mediante bases de datos orientadas a grafos permite **identificar relaciones explícitas e implícitas** entre usuarios y grupos de usuarios, así como identificar la forma en la que ellos interactúan pudiendo inferir el comportamiento de un usuario en base a sus conexiones. En una red social, dos usuarios presentan una **conexión explícita** si están directamente conectados. Esto ocurre, por ejemplo, entre dos usuarios de Facebook que son amigos o dos compañeros de trabajo de la misma empresa en LinkedIn. Por otra parte, una **relación implícita** es aquella que se produce entre dos usuarios a través de un intermediario, como puede ser otro usuario, un post donde han hecho un comentario, un “like” o un artículo que ambos han comprado.
- **Sistemas de recomendación:** Estos sistemas permiten modelar en forma de grafo las relaciones que se establecen entre personas o usuarios y cosas, como pueden ser productos, servicios, contenido multimedia o cualquier otro concepto relevante en función del dominio de aplicación. **Las relaciones se establecen en función del comportamiento de los usuarios al comprar, consumir contenido, puntuarlo o evaluarlo etc.** De esta forma, los sistemas de recomendación **identifican recursos de interés** para un usuario específico y pueden **predecir su comportamiento al comprar un producto o contratar un servicio**.

- **Información geográfica:** Se trata del caso de aplicación más popular de la teoría de grafos. Las aplicaciones de las bases de datos orientadas a grafos en información geográfica van desde **calcular rutas óptimas entre dos puntos en cualquier tipo de red** (red de carreteras, de ferrocarril, aérea, logística...) **hasta encontrar todos los puntos de interés en un área concreta, encontrar el centro de una región u obtener la intersección entre dos o más regiones, entre otras muchas.** Así, las bases de datos orientadas a grafos permiten modelar estos casos de aplicación como grafos dirigidos sobre los cuales operar para obtener los resultados deseados.

Aunque en este apartado se hayan analizado las principales áreas de aplicación, son muchos los dominios de aplicación donde las bases de datos orientadas a grafos se están convirtiendo en una solución predominante, como lo son **la gestión de datos maestros, gestión de redes y centros de datos o control de acceso entre otros muchos.**

4.2. Neo4j

En esta sección se profundiza en una tecnología concreta de bases de datos orientadas a grafos. Aunque existe un amplio abanico de tecnologías que dan soporte a este tipo de bases de datos, como lo son **OrientDB**¹, **FlockDB**² o **Infinite-Graph**³, en esta sección se profundiza en el trabajo con bases de datos orientadas a grafos utilizando **Neo4j**⁴.

4.2.1. Introducción

Neo4j es una plataforma de bases de datos orientada a grafos lanzada en 2007 e implementada en Java. Entre sus características principales destacan **el almacenamiento y procesamiento de grafos nativo**, lo que la convierte en una alternativa potente para el trabajo en entornos reales, donde se requiere de estas características para mejorar el rendimiento y la escalabilidad de los sistemas. Además, a pesar de lo anterior, Neo4j también **permite trabajar con modelos de datos en forma de grafo de manera transaccional**, por lo que también es útil en sistemas transaccionales.

Junto a todo lo anterior, Neo4j dispone de **amplias librerías que permiten crear también modelos de aprendizaje automático sobre grafos**. Esta plataforma puede integrarse con múltiples lenguajes de programación como Python o C++, entre otros, y ofrece una serie de herramientas muy útiles para usuarios y desarrolladores. A continuación, se enumeran y detallan algunas de ellas:

¹<https://orientdb.org>

²<https://github.com/twitter-archive/flockdb>

³<https://objectivity.com/infinitegraph/>

⁴<https://neo4j.com>

- **Graph Data Science Library:** Se trata de la librería principal que contiene una gran cantidad de algoritmos de búsqueda sobre grafos además de modelos de aprendizaje automático. Se trata de algoritmos y modelos eficientemente programados y escalables para el trabajo con grafos.
- **Neo4j Bloom:** Es una aplicación de visualización y exploración de grafos que permite la visualización de estos desde distintas perspectivas y puntos de vista, ofreciendo así una herramienta visual muy útil para visualizar los resultados de los algoritmos así como mostrar resultados a clientes.
- **Cypher:** Es el lenguaje de creación de consultas utilizado en Neo4j. Se trata de un lenguaje inspirado en SQL, solo que más sencillo y optimizado para el trabajo con grafos.
- **Integradores:** Con el objetivo de integrar y conectar Neo4j con otras plataformas y lenguajes, se han desarrollado distintos conectores para integrar esta plataforma con tecnologías como Apache Spark, Apache Kafka, BI...
- **Herramientas para desarrolladores:** Neo4j dispone de versiones para escritorio y web así como un *sandbox* en el que se proporciona un entorno controlado e integrado de los servicios Neo4j para desarrolladores.
- **Neo4j Aura:** Neo4j ofrece un servicio de computación en la nube para la creación de grafos y ejecución de algoritmos y modelos sobre ellos. Se trata de Neo4j Aura, el cual se encuentra disponible de forma gratuita y está integrado con Google Cloud Platform.

Las características anteriormente mencionadas y la cantidad de herramientas disponibles que Neo4j pone a disposición de los desarrolladores han sido los motivos principales para optar por esta plataforma para ilustrar el trabajo con bases de datos orientadas a grafos.

4.2.2. Importación de datos. Creación de grafos

Para comenzar a trabajar con Neo4j, es necesario **crear un grafo o bien partir de un grafo ya existente** que se importa dentro del área de trabajo de Neo4j. A continuación, se muestra cómo implementar ambos procedimientos.

Importación de un grafo

Sea el caso en el que se pretende importar un grafo a partir de un fichero **.csv**. En el ejemplo que se muestra a continuación, se dispone de dos ficheros **.csv** que contienen, respectivamente, **la definición de los nodos y de las aristas del grafo que se pretende importar**. Así pues, el listado 4.1 muestra los nodos del grafo. Para cada nodo se almacena su identificador, latitud, longitud y población.

Listado 4.1: Definición de nodos de un grafo

```

1 id,latitude,longitude,population
2 "Amsterdam",52.379189,4.899431,821752
3 "Utrecht",52.092876,5.104480,334176
4 "Den Haag",52.078663,4.288788,514861
5 "Immingham",53.61239,-0.22219,9642
6 "Doncaster",53.52285,-1.13116,302400
7 "Hoek van Holland",51.9775,4.13333,9382
8 "Felixstowe",51.96375,1.3511,23689
9 "Ipswich",52.05917,1.15545,133384
10 "Colchester",51.88921,0.90421,104390
11 "London",51.509865,-0.118092,8787892
12 "Rotterdam",51.9225,4.47917,623652
13 "Gouda",52.01667,4.70833,70939

```

Por su parte, el listado 4.2 especifica cada uno de los enlaces entre los nodos del grafo. Para cada uno de los enlaces se identifica el origen y destino del enlace, el tipo de relación que se define entre ellos y el coste.

Listado 4.2: Definición de aristas de un grafo

```

1 src,dst,relationship,cost
2 "Amsterdam","Utrecht","EROAD",46
3 "Amsterdam","Den Haag","EROAD",59
4 "Den Haag","Rotterdam","EROAD",26
5 "Amsterdam","Immingham","EROAD",369
6 "Immingham","Doncaster","EROAD",74
7 "Doncaster","London","EROAD",277
8 "Hoek van Holland","Den Haag","EROAD",27
9 "Felixstowe","Hoek van Holland","EROAD",207
10 "Ipswich","Felixstowe","EROAD",22
11 "Colchester","Ipswich","EROAD",32
12 "London","Colchester","EROAD",106
13 "Gouda","Rotterdam","EROAD",25
14 "Gouda","Utrecht","EROAD",35
15 "Den Haag","Gouda","EROAD",32
16 "Hoek van Holland","Rotterdam","EROAD",33

```

Dados estos dos ficheros, es posible **importar los nodos del grafo** según se indica en el listado 4.3.

Listado 4.3: Importación de nodos

```

1 WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/transport-nodes.csv"
   AS uri
2 LOAD CSV WITH HEADERS FROM uri AS row
3 MERGE (place:Place {id:row.id})
4 SET place.latitude = toFloat(row.latitude),
5   place.longitude = toFloat(row.longitude),
6   place.population = toInteger(row.population)

```

Así, en el listado anterior se define como identificador de recurso la dirección donde se encuentra el archivo que se pretende cargar, el cual es cargado a través de la instrucción **LOAD CSV**. La instrucción **MERGE** permite definir cada fila como nodo de un grafo de tipo lugar (*place*) y añadir a cada nodo la propiedad *id* cuyo valor será el identificador de la fila en la que se encuentra cada nodo dentro del fichero csv. Finalmente, los valores de latitud y longitud se convierten en valores de tipo real y la población en un valor de tipo entero.

Análogamente a la importación de nodos, el listado 4.4 muestra el código necesario para la **importación del fichero que contiene los enlaces** entre los nodos del grafo.

Listado 4.4: Importación de aristas

```
1 WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/transport-relationships.csv" AS uri
2 LOAD CSV WITH HEADERS FROM uri AS row
3 MATCH (origin:Place {id: row.src})
4 MATCH (destination:Place {id: row.dst})
5 MERGE (origin)-[:EROAD {distance: toInteger(row.cost)}]->(destination)
```

La diferencia principal de este listado con respecto al anterior, es que en este se necesita identificar mediante la instrucción **MATCH** los nodos de origen y destino de cada uno de los enlaces, lo cual es posible mediante los atributos “src” y “dst” incluidos en el archivo .csv. Finalmente, se utiliza la sentencia **MERGE** para crear una relación entre “origin” y “destination” de tipo **EROAD** que contiene la propiedad “distance” cuyo valor es el coste que aparecía en el fichero .csv.

Creación de un grafo

Otra alternativa es la **creación de un grafo desde cero**, especificando para ello sus nodos y enlaces. La figura 4.2 muestra un ejemplo de un **grafo de co-ocurrencia de hashtags** en el que cada uno de los nodos es un hashtag y los enlaces entre ellos se producen cuando dos hashtags aparecieron en un mismo tweet. Sobre cada enlace, se puede apreciar un valor que indica en cuántos tweets concurrieron los hashtags conectados por medio de dicha arista.



Figura 4.2: Grafo de co-ocurrencia de hashtags

Para representar este grafo en Neo4j, se puede escribir el siguiente fragmento de código que muestra el listado 4.5.

Listado 4.5: Creación grafo de co-ocurrencia de hashtags

```

1 CREATE
2     (JS:Hashtag {name: 'JoaquinSabina'}),
3     (RS:Hashtag {name: 'Rusia2018'}),
4     (AG:Hashtag {name: 'Argentina'}),
5     (FD:Hashtag {name: 'Feliz Domingo'}),
6     (MS:Hashtag {name: 'Messi'}),
7
8     (JS)-[:C00C {ntweet: 52}]->(FD),
9     (FD)-[:C00C {ntweet: 52}]->(JS),
10    (RS)-[:C00C {ntweet: 183}]->(FD),
11    (FD)-[:C00C {ntweet: 183}]->(RS),
12    (RS)-[:C00C {ntweet: 73}]->(AG),
13    (AG)-[:C00C {ntweet: 73}]->(RS),
14    (AG)-[:C00C {ntweet: 112}]->(MS),
15    (MS)-[:C00C {ntweet: 112}]->(AG),
16    (FD)-[:C00C {ntweet: 81}]->(MS),
17    (MS)-[:C00C {ntweet: 81}]->(FD)

```

Tal y como se puede ver, en primer lugar se definen los nodos y sus propiedades y a continuación los enlaces y sus propiedades. **Todo grafo en Neo4j tiene aristas dirigidas, si bien es cierto que cuando se ejecutan las consultas puede no tenerse en cuenta la dirección de las aristas.** Por este motivo, se han especificado las aristas en ambas direcciones. En caso de querer crear una arista que sea interpretada como no dirigida, se debe crear sin el operador flecha que determina el sentido de la arista. Esto es posible hacerlo solo fuera de la instrucción **CREATE**. El listado 4.6 muestra cómo crear la primera relación del grafo solo que de forma no dirigida.

Listado 4.6: Creación de una arista no dirigida

```

1 MATCH(JS:Hashtag{name:'JoaquinSabina'})
2 MATCH(FD:Hashtag{name:'Feliz Domingo'})
3 MERGE (JS)-[:C00C{ntweet:52}]- (FD)

```

Para **mostrar el grafo**, es posible utilizar el siguiente comando. La figura 4.3 muestra cómo quedaría el grafo creado en el listado 4.5.

```
MATCH(n) return (n)
```

Para **eliminar un nodo** creado, es posible utilizar el comando:

```
MATCH(JS:Hashtag{name:'Joaquín Sabina'}) delete (JS)
```

Sin embargo, **un nodo solo puede ser eliminado si se eliminan las relaciones que contiene.** Para ello, es posible utilizar el comando:

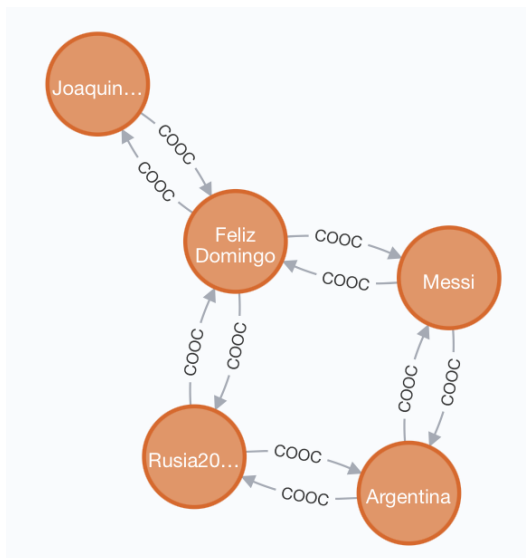


Figura 4.3: Visualización del grafo de co-ocurrencia de hashtags

```
MATCH(JS:Hashtag{name:'Joaquín Sabina'}) detach delete (JS)
```

Mientras que, **para eliminar todos los nodos y aristas**, es posible escribir:

```
MATCH(n) detach delete(n)
```

4.2.3. Recorridos sobre grafos

Los recorridos sobre grafos son una utilidad imprescindible en el trabajo con esta estructura de datos. Del mismo modo, los recorridos también son una herramienta fundamental cuando se trabaja con bases de datos orientadas a grafos.

Un recorrido sobre grafos es un procedimiento sistemático que permite explorar un grafo examinando todos sus vértices y aristas, comenzando desde un vértice inicial. A la hora de recorrer un grafo, existen dos tipos de recorridos: el **recorrido en anchura** (denotado BFS, por sus siglas en inglés) y el **recorrido en profundidad** (denotado DFS, también por sus siglas en inglés).

Recorrido en anchura (BFS)

El recorrido en anchura, también conocido por sus siglas en inglés *Breadth First Search* (BFS) es un procedimiento que permite recorrer un grafo, explorando todos sus nodos y aristas.

El recorrido en anchura parte de un nodo inicial. A continuación, el método comienza a explorar todos los nodos hijo del nodo inicial seleccionado que se encuentran a un salto. Una vez visitados, comienzan a explorarse los nodos hijo del siguiente nivel y así sucesivamente hasta haber recorrido el grafo completo. Este recorrido es muy utilizado cuando se busca un camino con el número mínimo de aristas entre dos vértices dados o cuando se busca un ciclo simple, es decir, una secuencia de nodos y aristas que se pueden describir circularmente en el que todos los nodos y aristas son distintos.

Por ejemplo, sea el grafo de la figura 4.4 que se corresponde con el grafo que se importó anteriormente. Es posible recorrerlo utilizando BFS escribiendo el siguiente código Neo4j que aparece en el listado 4.7.

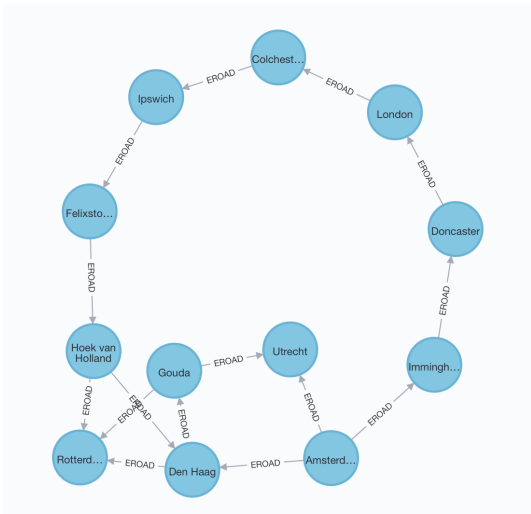


Figura 4.4: Visualización del grafo de ciudades

Listado 4.7: Recorrido en anchura

```
1 CALL gds.graph.create('myGraph', 'Place', 'EROAD', {relationshipProperties: 'distance'})
2 MATCH (a:Place{id:'Doncaster'})
3 WITH id(a) AS startNode
4 CALL gds.alpha.bfs.stream('myGraph', {startNode: startNode})
5 YIELD path
6 UNWIND [ n in nodes(path) | n.id ] AS tags
7 RETURN tags
```

En el fragmento anterior, se define un grafo llamado *myGraph* el cual está formado por nodos de tipo *Place* y cuyas relaciones son del tipo **EROAD** y tienen una propiedad, denominada *distance*. A continuación, se define el nodo inicial con las sentencias **MATCH** y **WITH**. Después, se llama al método que realiza el recorrido BFS sobre el grafo anterior partiendo desde el nodo inicial para producir

un camino (*path*). Finalmente, se devuelven todos los nodos que forman parte del camino, mostrando los identificadores de cada uno de ellos. El resultado del recorrido es: Doncaster, London, Colchester, Ipswich, Felixstowe, Hoek van Holand, Den Haag, Rotterdam, Gouda, Utrecht.

Recorrido en profundidad (DFS)

El recorrido en profundidad, también conocido por sus siglas en inglés *Depth First Search* (DFS) es un procedimiento alternativo al recorrido en anchura para explorar un grafo, recorriendo todos sus nodos y aristas.

El procedimiento para recorrer un grafo mediante el método DFS es el siguiente: en primer lugar, se parte de un nodo inicial. A partir de él, se empiezan a recorrer todos sus hijos hasta el último nivel. Una vez llegados al último nivel del grafo, se vuelve hacia atrás recorriendo todos los hijos de los nodos de niveles anteriores.

En Neo4j, el procedimiento para recorrer un grafo utilizando el recorrido BFS se muestra en el listado de código 4.8. El procedimiento, como se puede comprobar, es prácticamente idéntico al del listado 4.7 solo que cambiando el método de recorrido. El resultado obtenido es: Doncaster, London, Colchester, Ipswich, Felixstowe, Hoek van Holland, Rotterdam, Den Haag, Gouda, Utrecht.

Listado 4.8: Recorrido en profundidad

```
1 CALL gds.graph.create('myGraph','Place','EROAD',{relationshipProperties:'distance'})
2 MATCH (a:Place{id:'Doncaster'})
3 WITH id(a) AS startNode
4 CALL gds.alpha dfs.stream('myGraph', {startNode: startNode})
5 YIELD path
6 UNWIND [ n in nodes(path) | n.id ] AS tags
7 RETURN tags
```

4.2.4. Caminos mínimos

El cálculo y la obtención de caminos mínimos dentro de un grafo es uno de los problemas clásicos en teoría de grafos con multitud de aplicaciones en el mundo real. **Un camino dentro de un grafo es un conjunto de nodos y aristas que conectan un par de nodos del grafo.** En el caso de los **grafos no pesados**, es decir, aquellos cuyas aristas no tienen coste, el cálculo del camino mínimo se obtiene **minimizando el número de saltos** entre el nodo origen y el nodo destino. En el caso de los **grafos pesados**, aquellos cuyas aristas tienen coste, el cálculo del camino mínimo se obtiene a través del camino en el que **la suma de los costes de sus aristas es mínima**.

Junto con los recorridos de grafos, los métodos de obtención de caminos mínimos son muy útiles cuando se trabaja en **entornos dinámicos**, donde aparecen y desaparecen continuamente aristas del grafo o sus costes cambian dinámicamente. El cálculo de caminos mínimos también es muy útil en aplicaciones que deben dar

respuestas en tiempo real. Algunos ejemplos claros pueden ser encontrar rutas entre dos localidades, como ocurre en Google Maps u obtener los grados de separación entre una empresa y un empleado potencial al que se quiere contratar, como puede ocurrir en LinkedIn.

El **algoritmo de Dijkstra** es uno de los algoritmos más utilizados en teoría de grafos para la obtención de caminos mínimos. Dado el grafo de ciudades con el que se está trabajando a lo largo de este capítulo, el listado 4.9 permite obtener **los caminos mínimos entre un nodo del grafo y todos los demás nodos alcanzables.**

Listado 4.9: Ejecución del algoritmo de Dijkstra sobre el nodo Amsterdam

```

1 MATCH (source:Place{id:'Amsterdam'})
2 CALL gds.allShortestPaths.dijkstra.stream('myGraph', {
3   sourceNode: source,
4   relationshipWeightProperty: 'distance'
5 })
6 YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
7 RETURN
8   index,
9   gds.util.asNode(sourceNode).id AS sourceNodeName,
10  gds.util.asNode(targetNode).id AS targetNodeName,
11  totalCost,
12  [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
13  costs,
14  nodes(path) as path
15 ORDER BY index

```

En el listado anterior, se utiliza la sentencia **MATCH** para definir el nodo de origen y, posteriormente, se guarda el grafo incluyendo el nodo de origen y la propiedad de distancia que es la utilizada para obtener el camino mínimo. A continuación, se producirá un conjunto de caminos en el que se mostrará información completa del nodo de origen, destino, coste total y camino recorrido así como los distintos costes parciales. En caso de querer obtener el camino mínimo entre un par de nodos concretos, es posible utilizar el código mostrado en el listado 4.10 análogo al listado 4.9.

Listado 4.10: Ejecución del algoritmo de Dijkstra sobre el par de nodos Amsterdam-London

```

1 MATCH (source:Place{id:'Amsterdam'}), (target:Place{id:'London'})
2 CALL gds.shortestPath.dijkstra.stream('myGraph', {
3   sourceNode: source,
4   targetNode: target,
5   relationshipWeightProperty: 'distance'
6 })
7 YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
8 RETURN
9   index,
10  gds.util.asNode(sourceNode).name AS sourceNodeName,
11  gds.util.asNode(targetNode).name AS targetNodeName,
12  totalCost,
13  [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
14  costs,

```

```

15     nodes(path) as path
16 ORDER BY index

```

El camino mínimo entre Amsterdam y Londres es el formado por: Amsterdam Immingham, Doncaster, London. Este camino tiene un coste total de 720.

El **método A*** es una **técnica de búsqueda informada** que permite, al igual que el algoritmo de Dijkstra, **calcular el camino mínimo entre dos nodos dados cualesquiera**. Al contrario que el algoritmo de Dijkstra, cuando se va a seleccionar el siguiente nodo del camino, la técnica A* no solo se tiene en cuenta la distancia ya calculada, sino que **este algoritmo combina la distancia calculada con el resultado de una función heurística**. Esta función toma un nodo como entrada y devuelve un valor que corresponde con el coste de llegar al nodo objetivo desde ese nodo. Esta técnica, que se conoce también como de **búsqueda informada**, continúa el recorrido del grafo en cada iteración desde el nodo con el menor coste combinado entre la distancia ya calculada y la función heurística. El listado 4.11 muestra cómo obtener el camino mínimo mediante el método A* entre Amsterdam y Londres.

Listado 4.11: Ejecución del algoritmo A* sobre el par de nodos Amsterdam-London

```

1 CALL gds.graph.create(
2     'myGraph2',
3     'Place',
4     'EROAD',
5     {
6         nodeProperties: ['latitude', 'longitude'],
7         relationshipProperties: 'distance'
8     }
9 )
10 MATCH (source:Place{id:'Amsterdam'}), (target:Place{id:'London'})
11 CALL gds.shortestPath.astar.stream('myGraph2', {
12     sourceNode:source,
13     targetNode:target,
14     latitudeProperty:'latitude',
15     longitudeProperty:'longitude',
16     relationshipWeightProperty: 'distance'
17 })
18 YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
19 RETURN
20     index,
21     gds.util.asNode(sourceNode).id AS sourceNodeName,
22     gds.util.asNode(targetNode).id AS targetNodeName,
23     totalCost,
24     [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
25     costs,
26     nodes(path) as path
27 ORDER BY index

```

Como se puede ver, este fragmento de código es similar a los vistos anteriormente, solo que a la hora de guardar el grafo, se almacenan las propiedades de latitud y longitud, necesarias en Neo4j para ejecutar el método A*.

4.2.5. Medidas de centralidad

La centralidad se define como la relevancia de un nodo dentro de un grafo. Por tanto, el estudio de medidas de centralidad permite **identificar nodos relevantes dentro de un grafo**. Esta identificación permitirá entender el comportamiento de la red, qué nodos permiten viralizar con mayor rapidez el contenido de la red, desde qué nodos la información está más accesible, etc. El estudio de medidas de centralidad tiene multitud de aplicaciones, aunque son especialmente notables las relacionadas con los ámbitos de la **publicidad y el marketing**, donde el estudio de estas métricas permite identificar actores relevantes dentro de una red a los que se puede contratar para anunciar un producto o identificar sobre qué actores enviar información para alcanzar a más usuarios.

Respecto a las medidas de centralidad, existe una única medida sino que, en función de los datos y del propósito que se pretende alcanzar, se utilizan unas métricas u otras. a continuación, se van a definir tres medidas de centralidad con las que se realizarán ejemplos en Neo4j: **centralidad de grado, cercanía e intermediación**.

Para trabajar con medidas de centralidad, se va a importar un grafo social que servirá de ejemplo para el cálculo de estas métricas. El listado 4.12 muestra el código necesario para su creación. Por otra parte, la figura 4.5 muestra una visualización del grafo importado.

Listado 4.12: Importación de un grafo social

```
1 WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/" AS base
2 WITH base + "social-nodes.csv" AS uri
3 LOAD CSV WITH HEADERS FROM uri AS row
4 MERGE (:User {id: row.id})
5
6 WITH base + "social-relationships.csv" AS uri
7 LOAD CSV WITH HEADERS FROM uri AS row
8 MATCH (source:User {id: row.src})
9 MATCH (destination:User {id: row.dst})
10 MERGE (source)-[:FOLLOWS]->(destination)
```

Centralidad de grado

El grado de un nodo en un grafo se define como el número de aristas o conexiones que entran o salen de un nodo. Así pues, en un nodo de un grafo se distinguen dos grados: el **grado de entrada**, correspondiente al conjunto de conexiones que entran a un nodo y el **grado de salida**, que se corresponde con el conjunto de aristas que salen de un nodo. El primero indica la **prominencia** de un nodo dentro de la red, mientras que el segundo la **influencia** del nodo dentro del grafo. Esta medida de centralidad es muy utilizada para identificar actores prominentes o influyentes o para identificar conductas fraudulentas, caracterizadas en ocasiones por una actividad anormal. Para calcular la centralidad de grado en este

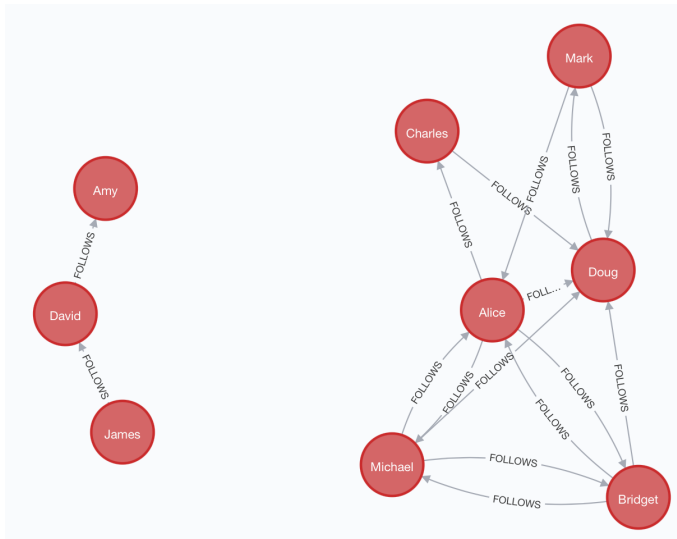


Figura 4.5: Visualización de un grafo social

grafo, se almacenará en primer lugar el grafo creado y, a continuación, se ejecutará el método que calcula la centralidad de grado según el código mostrado en el listado 4.13. El nodo con mayor grado es Doug, que tiene un grado de 5, seguido de Alice que tiene un grado de 3.

Listado 4.13: Cálculo de la centralidad de grado

```

1 CALL gds.graph.create(
2   'myGraph',
3   'User',
4   {
5     FOLLOWS: {
6       orientation: 'REVERSE'
7     }
8   }
9 )
10 CALL gds.degree.stream('myGraph')
11 YIELD nodeId, score
12 RETURN gds.util.asNode(nodeId).id AS name, score AS followers
13 ORDER BY followers DESC, name DESC

```

Cercanía

La cercanía es una medida de centralidad que determina qué nodos del grafo expanden rápida y eficientemente la información a través del grafo. Para calcular esta medida, se obtiene **la suma del inverso de las distancias de un nodo al resto**. De esta forma, dado un nodo u la cercanía del mismo se calcula por medio de la ecuación (4.1).

$$C(u) = \frac{1}{\sum_{v=1}^{n-1} d(u, v)} \quad (4.1)$$

Donde n es el número de nodos del grafo y $d(u, v)$ es la distancia del camino mínimo entre u y v . **La cercanía es una métrica muy utilizada para estimar tiempos de llegada en redes logísticas, para descubrir actores en posiciones privilegiadas en redes sociales o para estudiar la prominencia de palabras en un documento en el campo de la minería de textos.** Para obtener la cercanía en el grafo social de ejemplo, se puede utilizar el fragmento de código mostrado en el listado 4.14. En el grafo de la imagen, Alice, Doug y David tienen una cercanía de 1.

Listado 4.14: Cálculo de la cercanía

```
1 CALL gds.alpha.closeness.stream({
2   nodeProjection: 'User',
3   relationshipProjection: 'FOLLOWS'
4 })
5 YIELD nodeId, centrality
6 RETURN gds.util.asNode(nodeId).name AS user, centrality
7 ORDER BY centrality DESC
```

Intermediación

La intermediación es una medida de centralidad que permite detectar la influencia que tiene un nodo o actor del grafo en el flujo de información o de recursos de la red. El cálculo de la intermediación permite identificar a nodos que hacen de puentes entre distintas porciones del grafo. Esta medida de centralidad **es muy utilizada para la identificación de influencers y el estudio de la viralización de mensajes en redes sociales.** Intuitivamente, la intermediación de un nodo será mayor en tanto en cuanto dicho nodo aparezca en los caminos mínimos de cualquier otro par de nodos. Más formalmente, la intermediación puede calcularse según la ecuación (4.2).

$$B(u) = \sum_{s \neq u \neq t} \frac{p(u)}{p} \quad (4.2)$$

Donde u es el nodo del cual se calcula la intermediación (B), s y t son nodos del grafo, $p(u)$ es el número de caminos mínimos entre s y t que pasan por u y p es el número de caminos mínimos entre s y t . Para el cálculo de la intermediación, se puede utilizar el fragmento de código mostrado en el listado 4.15. Al ejecutar este listado, se obtiene que Alice tiene la mayor intermediación, teniendo un valor de 10.

Listado 4.15: Cálculo de la intermediación

```
1 CALL gds.betweenness.stream('myGraph')
2 YIELD nodeId, score
3 RETURN gds.util.asNode(nodeId).name AS name, score
4 ORDER BY score DESC
```

4.2.6. Detección de comunidades

A la hora de trabajar con grafos reales, que presentan una gran cantidad de nodos y enlaces, muchas veces se pretende **identificar comunidades** dentro del grafo para aplicar algoritmos sobre ellas. **Una comunidad es, por tanto, un conjunto de nodos que presentan más relaciones entre sí que con el resto de nodos fuera de la comunidad.** La detección e identificación de comunidades permite **identificar comportamientos emergentes y de rebaño dentro de una red.** De esta forma, es posible detectar y predecir hábitos de los usuarios. A continuación, se van a aplicar tres métodos de detección de comunidades sobre el grafo social de ejemplo que se ha utilizado en secciones anteriores.

Conteo de triángulos

Un triángulo es un conjunto de tres nodos que tienen relaciones entre sí. El conteo de triángulos para un nodo dado, permite estudiar o inspeccionar de forma global un grafo y, aplicado sobre componentes conexas, permite inspeccionar regiones de un grafo. Para aplicar este método, es necesario almacenar el grafo teniendo en cuenta que las aristas deben almacenarse como **UNDIRECTED**. El listado 4.16 muestra el fragmento de código necesario para aplicar este método sobre el grafo social de ejemplo. La ejecución de este método da como resultado que Alice y Doug son aquellos que pertenecen a más triángulos, un total de 5.

Listado 4.16: Ejecución del conteo de triángulos

```
1 CALL gds.graph.create(
2   'myGraph2',
3   'User',
4   {
5     FOLLOWS: {
6       orientation: 'UNDIRECTED'
7     }
8   }
```

```

9 )
10 CALL gds.triangleCount.stream('myGraph')
11 YIELD nodeId, triangleCount
12 RETURN gds.util.asNode(nodeId).name
13 AS name, triangleCount
14 ORDER BY triangleCount DESC

```

Coeficiente local de clustering

Este coeficiente **proporciona una medida cuantitativa del grado de agrupación de un nodo**. Para el cálculo del coeficiente local de clustering se utiliza el conteo de triángulos, de forma que se compara el grado de agrupación de un nodo con el grado máximo de agrupación que podría tener. Así pues, el coeficiente local de clustering se puede calcular mediante la ecuación (4.3).

$$CC(u) = \frac{2R_u}{k_u(k_u - 1)} \quad (4.3)$$

Donde u es un nodo, $R(u)$ es el número de relaciones que a través de los vecinos de u (lo cual puede medirse con el número de triángulos que pasan por u) y $k(u)$ es el grado de u . El cálculo del coeficiente local de clustering puede realizarse por medio del listado 4.17. Los actores Bridget, Charles, Mark y Michael tienen el mayor coeficiente local de clustering, que es 1.

Listado 4.17: Cálculo del coeficiente local de clustering

```

1 CALL gds.localClusteringCoefficient.stream('myGraph2')
2 YIELD nodeId, localClusteringCoefficient
3 RETURN gds.util.asNode(nodeId).id
4 AS name, localClusteringCoefficient
5 ORDER BY localClusteringCoefficient DESC

```

Por último, **el coeficiente global de clustering se calcula como la suma normalizada de los coeficientes de clustering locales**. Así, estos coeficientes nos permiten encontrar medidas cuantitativas para detectar comunidades, pudiendo especificar incluso un umbral para establecer la comunidad (por ejemplo, especificar que los nodos han de estar conectados en un 40 %).

Componentes fuertemente conexas

En un grafo dirigido, una componente fuertemente conexa es aquel grupo de nodos en el que cualquier nodo puede ser alcanzado por cualquier otro en ambas direcciones. El estudio de las componentes fuertemente conexas en un grafo permite **estudiar la conectividad de la red**. Para realizar el cálculo de las

componentes conexas, el cual se realiza en un tiempo de procesamiento proporcional al número de nodos, es posible utilizar el listado 4.18. El resultado permite comprobar que hay un total de cuatro componentes fuertemente conexas en el grafo social.

Listado 4.18: Obtención de componentes fuertemente conexas

```
1 CALL gds.alpha.scc.stream({
2   nodeProjection: 'User',
3   relationshipProjection: 'FOLLOWS'
4 })
5 YIELD nodeId, componentId
6 RETURN gds.util.asNode(nodeId).id AS Name, componentId AS Component
7 ORDER BY Component DESC
```

4.2.7. Predicción de enlaces

Los grafos son estructuras de datos que representan **sistemas dinámicos**, que evolucionan a lo largo del tiempo. Por este motivo, es muy común que en un grafo cualquiera aparezcan y desaparezcan nuevos nodos y conexiones entre dichos nodos. **Los métodos de predicción de enlaces permiten predecir qué enlaces se formarán próximamente entre los nodos del grafo, permitiendo adelantarse a los acontecimientos y prevenir eventualidades.** Como norma general, los métodos de predicción de enlaces **se basan en medidas de cercanía y de centralidad** asumiendo que los nuevos enlaces se producirán, mayoritariamente, sobre/desde los nodos más relevantes. A continuación, se detalla el funcionamiento de tres métodos comúnmente utilizados en predicción de enlaces.

Vecinos comunes

El método de vecinos comunes se basa en la idea genérica de que dos actores de la red que tienen una relación con un usuario común tendrán más posibilidad de conectarse entre sí que quienes no. Formalmente, dados dos nodos, la posibilidad de que se produzca un enlace entre los nodos x e y viene dada por la ecuación 4.4.

$$CN(x, y) = |N(x) \cap N(y)| \tag{4.4}$$

Donde $N(x)$ es el conjunto de nodos adyacentes a x y $N(y)$ es el conjunto de nodos adyacentes a y . Cuanto mayor es el valor de CN calculado, existe mayor posibilidad de que se produzca un nuevo enlace entre x e y . En el grafo social de ejemplo, el cálculo de vecinos comunes para Charles y Bridget se puede realizar a través del listado 4.19. El resultado de este cálculo es 2.

Listado 4.19: Obtención del valor de vecinos comunes para Charles y Bridget

```
1 MATCH (x:User{id:'Charles'})
```

```

2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.commonNeighbors(x,y) AS score

```

Adhesión preferencial

Este método se basa en la idea general de que cuanto más conectado está un nodo, es más probable que reciba nuevos enlaces. Formalmente, esta idea se puede expresar por medio de la ecuación 4.5

$$CN(x, y) = |N(x) \cdot N(y)| \quad (4.5)$$

En el grafo social de ejemplo, el cálculo de adhesión preferencial para Charles y Bridget se puede realizar a través del listado 4.20. El resultado de este cálculo es 10.

Listado 4.20: Obtención del valor de adhesión preferencial para Charles y Bridget

```

1 MATCH (x:User{id:'Charles'})
2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.preferentialAttachment(x, y) AS score

```

Asignación de recursos

Se trata de una métrica compleja que evalúa la cercanía de un par de nodos para determinar la posibilidad de que, entre ellos, se produzca un nuevo enlace. Para ello, se utiliza la expresión dada en la ecuación 4.6.

$$RA(x, y) = \sum_{U \in N(x) \cap N(y)} \frac{1}{N(u)} \quad (4.6)$$

En el grafo social de ejemplo, el cálculo de la asignación de recursos para Charles y Bridget se puede realizar a través del listado 4.21. El resultado de este cálculo es 0.309.

Listado 4.21: Obtención del valor de asignación de recursos para Charles y Bridget

```

1 MATCH (x:User{id:'Charles'})
2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.resourceAllocation(x, y) AS score

```

Bibliografía

- [AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.
- [BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.
- [BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms*. Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.
- [dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.
- [FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.
- [Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.
- [GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.
- [HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

- [IGG03] C. Imhoff, N. Galemme, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.
- [Inm02] William H. Inmon. *Building the Data Warehouse, 3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.
- [JVL03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.
- [KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.
- [MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.
- [MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.
- [PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma, 2006.
- [RB02] Pablo Rodríguez Bocca. *Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos*. Interoperabilidad, Monografía, 2002.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.
- [SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.
- [Tiw11] Shashank Tiwari. *Professional NoSQL*. Wrox Press Ltd., 2011.
- [TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.